RESEARCH NOTE / LEAN 4.32.0

Three further proof simplifications

Shape balance, ordered windows, and full-exponent growth

For Vladimir Reshetnikov's ProveIt repository

26 October 2026

Scope. Existing proofs in `Computability/DiophantineConsistency`, after treating the earlier proposals as already available. The final exclusion corpus contains 43 placed packets, including the five initially incoming archives.

Repository snapshot. `9d9d1123b39927d30363f1dcd53c23f0575c30ee`.

Freshness cutoff. `60fcb2a69b7f1fdd4c3592ea5f164223cad50abe`, committed at 19:10:07 UTC on 26 October. The direct target files are unchanged.

Main result. Raw signed-expression splitting produces two natural-expression trees with identical operation-labelled shapes after leaf labels are erased. This gives a scalar replacement for paired size metadata. A second proposal shares index uniqueness for monotone half-open windows between square pairing and division. A third removes a growth-proof detour in the newly added constructed-rank module.

Evidence discipline. The packet distinguishes checked proof cores, complete candidate source files, patch applicability, and remaining integration work. The ordered-window proposal increases the marginal source total by 16 lines and is offered for structural simplification. It is not counted as a line reduction.

This note proposes local proof and metadata changes. It makes no claim of a smaller Diophantine equation, fewer equation variables, a new consistency result, or a measured compilation speedup.

Contents

1 Findings and recommended order **2**

2 Baseline and non-repetition audit **2**

2.1 What was treated as already available . . . 2

2.2 The exact novelty boundary . . . 3

2.3 Rejected alternatives and audit limits . . . 3

3 A: exploit exact balance of the two split trees **4**

3.1 The current translation . . . 4

3.2 A theorem for every shape algebra . . . 4

3.3 Useful consequences . . . 5

3.4 Collapse the reachable metadata to one number . . . 5

3.5 Compatibility and integration requirements . . . 6

3.6 Where the theorem stops applying . . . 6

4 B: share uniqueness of monotone windows **6**

4.1 The common proof obligation . . . 6

4.2 The residual square-pairing application . . . 7

4.3 Division as reuse evidence . . . 7

4.4 Honest source accounting . . . 8

5 C: apply the growth theorem at the full exponent **8**

5.1 The exact current endpoint . . . 8

5.2 The replacement . . . 8

5.3 Why this is a valid residual change . . . 9

6 Validation and what remains to be checked **9**

7 Integration sequence and further questions **10**

7.1 Suggested sequence . . . 10

7.2 Further research questions . . . 10

Sources and artifact map **11**

1 Findings and recommended order

The strongest new structural observation is an invariant of the *existing* executable translation `Expr.splitNatural`. Its positive and negative trees can have different values, coefficients, and variable occurrences, but their shapes agree exactly after erasing every leaf label. In particular, both trees have the same node count and depth. The current two-component size recurrence therefore visits only diagonal pairs.

The smallest immediate edit is the full-exponent growth proof. It replaces a chain through positivity of a window, a lower bound on its radix, and a lower bound on the radix square by the universal inequality $n < 2^n$ at the exponent actually present in the final cube.

The ordered-window interface is useful when a common mathematical explanation is more valuable than immediate source reduction. It removes a duplicated index-comparison argument from square pairing, but its complete helper costs more lines than that one remaining client saves. Prior generic division-uniqueness work is already assumed applied and contributes no fresh saving here.

Direction	Concrete change	Recommendation
A: Shape	Prove a polymorphic fold equality; use a scalar recurrence and retain the public pair at the boundary.	The main reusable invariant. Test metadata consumers before adoption.
B: Windows	Recover a unique index from adjacent monotone windows; use it after the prior square-shell improvement.	Optional architecture. The measured post-prior package grows by 16 lines.
C: Growth	Use $2u \le 3u < 2^{3u} = q^3$ in the constructed-rank theorem.	First small patch to integrate: five physical lines removed, no helper.

The supplied numerical counts refer to physical source lines in explicitly identified files. They are not estimates of proof-term size or execution cost. They also do not purport to measure a hypothetical repository obtained by merging every historical proposal, because those proposals are not all a mutually compatible patch series.

2 Baseline and non-repetition audit

2.1 What was treated as already available

The review is pinned to the commit above, whose root configuration selects Lean 4.32.0 and Mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`. The maintained DiophantineConsistency module directory contains 519 Lean files and 81,677 physical lines at that snapshot. These figures include check modules and are not a measure of mathematical complexity.

The exclusion review covered all 36 placed packet directories, labelled a through zj, and the five ZIPs present in `docs/incoming`. Those ZIPs contain 1,933 non-directory members, including 1,899 searchable text members, and no nested ZIPs. The focused checkout materialized 2,068 research text/code files. Packet READMEs and manuscript proposal indexes were read, and candidate-specific comparisons included technical notes and rejected or exploratory ideas. The inventory and exact supporting paths are in `audit/`.

A final refresh at commit `60fcb2a69b7f1fdd4c3592ea5f164223cad50abe` found 43 placed packets, a through zq, with only a README remaining in `docs/incoming`. The original 36 packet trees are unchanged. Packets zk–zo relocate the five audited archives: all 328 retained substantive files match the original

archive members byte-for-byte. The two additional packets, zp and zq, were reviewed through 40 blob-verified text/code files, including reports, candidates, and exploratory notes. Their controller, subset-sum, reflected-source, and LTE proposals are also treated as already available. None covers the three narrow contributions retained here.

An untruncated comparison of all 519 maintained module blobs found five changes outside the direct targets and no added or removed modules. The six direct target files are byte-identical between the proof snapshot and final cutoff. The source comparison does not establish a fresh build of the newer dependency closure. Full evidence and refresh limits are in `audit/prior_art_refresh.md` and its JSON companion.

Already-covered families include scanner induction, decoder invariants, formula semantic specifications, witness transport, generated circuit proofs, common circuit tails, coefficient envelopes, sparse-sum naturality, Pell parity and growth arguments, source-row consequences, controller residue interfaces, periodicity, finite-order rigidity, and representation-depth certificates. Recommending one of those families again would not satisfy this iteration.

2.2 The exact novelty boundary

For A, the contribution is *exact shape balance and the resulting diagonal invariant*. A general suggestion to fold metadata or expose witness-count projections is prior work. In fact, the incoming additional-certificates taxonomy explicitly excludes merely moving or reusing count projections in `ExpressionSize`. The present theorem equates every leaf-insensitive structural interpretation of the two raw trees, including interpretations into noncommutative algebras.

For B, generic ordered-semiring division uniqueness is explicitly mentioned as an exploratory direction in the incoming additional-certificates taxonomy. It is excluded as a new endpoint. The retained proposal is the *shared monotone-boundary theorem* and its use in the square-pairing proof after packet y's shell-cut changes. Division illustrates reuse of the same interface. The paired applications count as one architectural idea.

For C, earlier exponential-threshold machinery is assumed available. The new contribution is a precise application to `CurrentPackingRank2011.rank_gt_twice_window`, introduced by commit `493d46d498`. This theorem concerns a constructed packing before source-zero and Pell hypotheses are available. Earlier source-rank bounds cannot simply be substituted for it.

2.3 Rejected alternatives and audit limits

The review rejected a direct `fourBlocks_cast` eliminant. Its apparent new polynomial identity is exactly the reversed identity already proved by `NativeSourceMask.packing_four_block`; packet zj also explicitly excludes the associated cast-cleanup direction. Direct elimination of the source rows giving $N = q^3$, a second fixed-point scanner abstraction, and joint simplifier correctness/cost induction were excluded for similar reasons.

This is a documented non-repetition review, not a proof that no equivalent thought occurs anywhere in the repository. Historical PDFs were not visually reread when their TeX sources were available. Some provenance TSVs, build configuration files, and checksum metadata were inventoried by path rather than fetched. Old proofs and logs were not all semantically replayed. Historical non-simplification dossiers linked from the large incoming README were not recursively audited. The final candidate matrix records these limits and 20 exact evidence records with hashes and pinned repository paths.

3 A: exploit exact balance of the two split trees

3.1 The current translation

The translation `Expr.splitNatural` represents an integer expression e by natural expressions (e^+, e^-) such that

$$e(v) = e^+(v) - e^-(v)$$

after casting the natural values to integers. At leaves, an integer constant becomes two natural constants; a variable becomes that variable and zero. Addition and subtraction combine or swap the two components. Multiplication uses

$$(pq)^+ = p^+q^+ + p^-q^-, \qquad (pq)^- = p^+q^- + p^-q^+.$$

Those expressions differ semantically. Nevertheless, their tree structure has a stronger invariant than the numerical upper bounds currently needed by the compiler.

3.2 A theorem for every shape algebra

Let A be any type, choose an element $\ell \in A$, and choose two completely arbitrary operations

$$a, m : A \times A \longrightarrow A.$$

Define F on natural-expression syntax by sending *both* constants and variables to ℓ, addition nodes to a, and multiplication nodes to m. Then

$$F(e^+) = F(e^-) \tag{1}$$

for every signed expression e.

No associativity, commutativity, cancellation, or order hypothesis is needed. This is significant: equality of sizes alone would hide how much structure survives. Taking A to be operation-labelled trees with one anonymous leaf shows that the two shapes themselves agree, including child order and the distinction between addition and multiplication.

The proof is structural induction. At leaves, the two interpretations are ℓ. At addition or subtraction, the induction hypotheses identify the corresponding child interpretations. At multiplication, write the common values for the two children as P and Q. Both outputs then have interpretation

$$a(m(P, Q), m(P, Q)).$$

This is syntactic congruence, so an algebraic normalizer is unnecessary.

The supplied Lean theorem has the following form:

```
theorem splitNatural_shapeFold_eq {σ A : Type*} (leaf : A)
    (add mul : A → A → A) (e : Expr σ) :
    NatExpr.shapeFold leaf add mul e.splitNatural.1 =
      NatExpr.shapeFold leaf add mul e.splitNatural.2 := by
  induction e <;> simp_all [splitNatural, NatExpr.shapeFold]
```

3.3 Useful consequences

Choose $A = \mathbb{N}$, $\ell = 1$, and both operations equal to $(x, y) \mapsto 1 + x + y$. Equation (1) gives

$$\mathrm{count}(e^+) = \mathrm{count}(e^-).$$

Choose instead $\ell = 0$ and both operations $(x, y) \mapsto 1 + \max(x, y)$. It gives

$$\mathrm{depth}(e^+) = \mathrm{depth}(e^-).$$

The count and depth declarations used by the extracted-definition check come from the pinned source. The bridge from either concrete recursion to the corresponding generic fold is another structural induction; the mathematical equality is not inferred from a finite sample of expressions.

The existing graph compiler satisfies a count identity of the form

$$\mathrm{witnessCount}(t.\mathrm{graphRep}) + 1 = \mathrm{count}(t).$$

Consequently the two raw graphs also have equal witness counts. This is a consequence of the shape invariant plus an existing identity. It is not a proposal to add another redundant witness-count framework.

Other valid instances include the number of addition nodes, the number of multiplication nodes, or a structural cost charging the same amount for every leaf. Operations for the two internal node kinds may differ. This makes the result usable beyond one particular numerical size recurrence.

3.4 Collapse the reachable metadata to one number

The current `ExpressionSize.splitSize` returns a record with `positive` and `negative` fields. Its leaf value is $(1, 1)$. Its addition and subtraction operations preserve diagonal pairs, while its multiplication operation returns a diagonal pair by construction. Hence every reachable state lies in the diagonal

$$D = \{(h, h) : h \in \mathbb{N}\}.$$

Define the scalar recurrence

$$\begin{aligned} H(c) = H(x) &= 1, \\ H(p + q) = H(p - q) &= 1 + H(p) + H(q), \\ H(pq) &= 3 + 2H(p) + 2H(q). \end{aligned}$$

Then both raw tree counts equal $H(e)$, and the old pair-valued result is exactly $(H(e), H(e))$.

The candidate retains the public record and its operations, including their behavior on arbitrary off-diagonal inputs. It changes only the expression fold to compute H and package the result at the boundary:

```
def halfSize {σ : Type*} : Expr σ → ℕ
  | .constant _ | .var _ => 1
  | .add p q | .sub p q => 1 + halfSize p + halfSize q
  | .mul p q => 3 + 2 * halfSize p + 2 * halfSize q

def splitSize {σ : Type*} (e : Expr σ) : SplitSize :=
  ⟨halfSize e, halfSize e⟩
```

The whole candidate `ExpressionSize` file has 81 physical lines, compared with 90 in the pinned version. That nine-line difference includes the scalar recurrence and the revised proof. It excludes the optional explanatory shape-theorem module and the validation-only legacy comparison. If those explanatory declarations are installed as additional production API, they must be charged separately. The line difference is also not claimed to arise solely from the invariant: the candidate uses a compact induction script for its correctness theorem.

3.5 Compatibility and integration requirements

This candidate changes a *computational metadata definition*, rather than only a theorem body. Its values agree with the previous implementation for every input; the extracted check proves this by retaining the original fold under a legacy namespace and establishing equality with the replacement. Equality of values is distinct from preservation of every old definitional reduction path.

This matters most for clients that explicitly unfold `ExpressionSize.splitSize`. Check `BoundedForallSize` and `BoundedForallDepth` after adoption. Rewriting through a public equation is generally more stable than depending on the arrangement of the old pair recurrence. Concrete reductions still calculate the same numbers, but some symbolic `rfl`, `change`, or unfolding scripts may require adjustment.

No claim is made that the metadata refactor speeds up compilation. That would require a controlled comparison of the actual consuming modules. The reliable benefit is that the representation of the invariant becomes explicit and only one structural count is propagated internally.

3.6 Where the theorem stops applying

The equality requires every leaf to receive the same interpretation. It does not equate semantic values, variable supports, polynomial degrees, or costs that depend on a literal's magnitude. For a variable x, the positive half is x and the negative half is zero; their polynomials and supports already differ.

The theorem also concerns *raw* splitting. Separate simplification may destroy the shape equality. For example, split $x \cdot x$ and simplify each half. The positive half becomes $x \cdot x$ and the negative half becomes zero. A consumer of simplified syntax must use its own cost theorem; it must not reuse raw shape balance after normalization.

4 B: share uniqueness of monotone windows

4.1 The common proof obligation

Several encodings locate an index by putting a value between two consecutive boundaries. Division locates a quotient in a multiple-of-divisor interval. The maximum coordinate of square pairing is located in a square shell. The underlying uniqueness argument does not depend on either multiplication or squares.

Let I be linearly ordered and C be preordered. Suppose

$$B : I \to C \quad \text{is monotone}, \qquad s : I \to I, \qquad a < b \Longrightarrow s(a) \le b.$$

If

$$B(a) \le x < B(s(a)), \qquad B(b) \le x < B(s(b)),$$

then $a = b$.

To prove it, assume $a < b$. The successor condition and monotonicity give

$$x < B(s(a)) \le B(b) \le x,$$

a contradiction. The opposite strict inequality is symmetric, and linearity finishes the proof.

The codomain needs no antisymmetry. The boundary map needs no strict monotonicity. The indices need no well-foundedness, external finiteness, or Archimedean property. These weak assumptions are useful for the repository's internal-natural constructions, where replacing an internal carrier with external `Nat` would be invalid.

```
theorem index_unique (succ : I → I)
    (step_le : ∀ {a b : I}, a < b → succ a ≤ b)
    (base : I → C) (mono : Monotone base)
    {a b : I} {x : C}
    (ha : base a ≤ x) (ha' : x < base (succ a))
    (hb : base b ≤ x) (hb' : x < base (succ b)) : a = b := by
  apply le_antisymm
  · exact le_of_not_gt (fun h =>
      (not_lt_of_ge ha) (hb'.trans_le (mono (step_le h))))
  · exact le_of_not_gt (fun h =>
      (not_lt_of_ge hb) (ha'.trans_le (mono (step_le h))))
```

4.2 The residual square-pairing application

Write $m = \max(a, b)$. The prior shell argument gives

$$m^2 \le \mathrm{pair}(a, b) < (m + 1)^2.$$

For two pairs with the same code, use the shared theorem with $B(m) = m^2$ and $s(m) = m + 1$. Monotonicity of squaring holds on the nonnegative semiring carrier. The needed successor comparison is supplied by the existing discrete-order bridge. Thus the two maxima agree.

Packet y's sharper `pair_lt_shell_cut` argument already handles the subsequent branch decision. The new candidate preserves that theorem and its two-branch argument. It replaces only the common-maximum step that previously compared two possible maxima by contradiction.

The main patch is deliberately rebased onto the *packet-y candidate*, not advertised as though its shell improvement were new. The exact prior file is included under `windows/prior_baseline/`, together with its hash. Preservation checks confirm that the shell-cut section and the later branch/internal-wrapper text are unchanged.

4.3 Division as reuse evidence

For division, set $B(q) = qd$ and $s(q) = q + 1$. From $n = qd + r$ and $0 \le r < d$ one obtains

$$qd \le n < (q + 1)d.$$

The window theorem identifies q; additive cancellation then identifies r. The compatible generic example is checked over an ordered semiring. It has no external-natural induction hypothesis.

Generic ordered-semiring division uniqueness itself is already discussed in prior material, so this report credits it with *zero* fresh reduction. It is included to explain why the new abstraction has more than one mathematical client, not to increase the headline count of new ideas.

4.4 Honest source accounting

Against packet y, the pairing module decreases from 100 to 94 lines. The complete helper module adds 22 lines, including its import, documentation, namespace, and theorem. The marginal package therefore grows from 100 to 116 lines: **16 additional lines**. The new import in the pairing candidate is included in that count.

This is an optional architectural choice. Adopt it when a common interval contract makes subsequent proofs clearer, or when another genuine residual client amortizes the helper. Keep the local proof if the immediate objective is only the fewest source lines. No conjectural future reuse is subtracted from the present total.

The strict upper boundary is essential. Replacing both upper inequalities by weak inequalities permits adjacent closed intervals to share an endpoint. Conversely, a constant boundary map simply yields empty strict windows; no special nondegeneracy premise is needed to keep the theorem correct.

5 C: apply the growth theorem at the full exponent

5.1 The exact current endpoint

In `CurrentPackingRank2011.rank_gt_twice_window`, put

$$u = \mathrm{width} \cdot N, \qquad q = 2^u.$$

The module has already proved a natural four-block lower bound $q^3 \le R$ and an integer cast equality identifying R with the constructed rank. The remaining numerical task is $2u < q^3$.

The current proof first obtains $u < q$, then establishes positivity of u, $2 \le q$, and $2 \le q^2$, and finally compares $2u$ with $2q$ and then q^3. This is correct, but the final expression already suggests a shorter argument.

5.2 The replacement

The universal natural inequality $n < 2^n$, applied at $n = 3u$, gives

$$2u \le 3u < 2^{3u} = (2^u)^3 = q^3.$$

The Lean proof uses the exponent ordering produced directly by `Nat.pow_mul`:

```
have hc : 2*(width*N) < q N^3 := by
  rw [q_binary, ← Nat.pow_mul]
  exact Nat.lt_of_le_of_lt (by omega)
    (Nat.lt_two_pow_self (n := (width*N)*3))
```

The library theorem's natural argument is implicit in the pinned Lean version. The named argument `(n := ...)` is required; a positional placeholder is not the checked code.

After this step, the original calls to the four-block lower bound and the cast equality finish the proof. The complete module decreases from 138 to 133 physical lines. There is no new helper or import in the proposed production patch.

5.3 Why this is a valid residual change

The arithmetic core even covers $u = 0$: $0 < 1$. It therefore avoids unnecessary positivity reasoning for this subgoal. The public theorem keeps its existing hypotheses because other parts of the constructed-packing interface use them; the patch does not claim those surrounding assumptions have all become redundant.

Earlier native-rank growth proposals concern ranks recovered from source equations and, in some cases, later Pell or positivity information. This endpoint precedes those obligations. The present proof uses only the exact binary-power identity and the already-proved constructive lower bound, so it does not create a circular dependence on the eventual source-system correctness.

This is a modest use of an existing library theorem, not a new exponential estimate. Its value is a smaller dependency chain in a newly added client.

6 Validation and what remains to be checked

All successful checks below used Lean 4.32.0, compiler commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The Mathlib-dependent checks used the pinned revision recorded in the repository.

Artifact	Successful checking scope
`ShapeCore.lean`	Hash-verified extraction of the actual syntax, splitting, count, depth, old metadata, and candidate metadata declarations. Proves the generic fold theorem, erased-tree equality, count/depth equality, candidate correctness, and equality to the original metadata for every expression.
`SplitShape.lean`	Complete repository-importing theorem module elaborated and emitted an object file. Both named shape theorems report only `propext`.
`ExpressionSize.lean`	The complete 81-line candidate elaborated and emitted an object file. This was a cached-prerequisite check, not a freshly rebuilt pinned dependency closure.
Window helper and pairing	Complete new 22-line helper compiled. The complete generic `MaxSquarePairing` namespace from the post-y candidate compiled; the internal-model wrapper was not rebuilt.
Growth core	Standard-library arithmetic core compiled, including the final natural-to-integer transport with explicit lower/cast hypotheses and zero/one exponent examples. The complete packing-rank module was not rebuilt.

The shape extractor checks SHA-256 hashes before extracting declarations. It changes only the spellings `ℕ` to `Nat`, `ℤ` to `Int`, and `Type*` to `Type _`, and places the original metadata function under a legacy namespace. Its comparison theorem is universal, rather than a test on a finite selection of examples. The generated file and its manifest are included, so extraction can be reproduced independently.

The repository-importing shape checks reused compiled prerequisites. Source comparison found the two cached files `Representation` and `QuarticCompiler` predated existing assignment-proof refactors at the pinned commit. The pertinent datatype, split, and count definitions agree; the extracted-

definition checker independently uses the exact pinned files. These observations support the stated core/candidate checks but do not establish a fresh build of the full dependency tree.

The generic shape, count, and depth equalities use only `propext`. The scalar correctness and legacy-equality checks use `propext` and `Quot.sound`; the new positive/negative projection equality is axiom-free. The window helper uses `propext`; the generic pairing checks report the standard set `propext`, `Classical.choice`, and `Quot.sound`. The growth core uses `propext` and `Quot.sound`. None of the delivered successful theorem audits reports `sorryAx`.

The shape and growth patches pass applicability checks against the pinned source. The window patch passes against its included exact post-y baseline. No full repository build, fresh full-baseline axiom comparison, controlled timing comparison, or simultaneous integration of all historical proposals is claimed.

Kernel-checked generic theorems and an applicable patch do not by themselves establish that all consumers in a large development rebuild. In particular, the scalar metadata change can alter symbolic reduction paths, and the ordered-window patch is based on the supplied post-y file rather than a fictional merge of all prior submissions.

The evidence files retain source hashes, compiler versions, logs, and the exact scope of successful checks. A successful generic core is described as a core check. A stopped exploratory invocation is not converted into a successful build claim. No benchmark or whole-repository success is inferred from these results.

7 Integration sequence and further questions

7.1 Suggested sequence

1. Apply the five-line growth reduction first. Rebuild `CurrentPackingRank2011` and its focused check module. It changes one theorem body and adds no shared API.
2. Review the scalar metadata fold and its proof of equality with the original fold. Rebuild `ExpressionSize`, then `BoundedForallSize` and `BoundedForallDepth`; inspect any client that unfolds the metadata definition. The generic shape theorem can be retained as a reusable interface or left as checked supporting evidence.
3. Consider the window abstraction separately. Apply it after packet y's shell-cut change. Use the included post-prior baseline and patch to avoid accidentally reintroducing old proof branches. Count its 16-line overhead explicitly.

The replay scripts distinguish standard-library cores from checks requiring the configured repository and Mathlib. They do not push commits or change a remote repository. Patch applicability checks are separate from compilation.

7.2 Further research questions

Which other paired metadata recurrences are constrained to a proper subset of their declared state type? The diagonal invariant here is stronger than a numerical estimate. Similar exact constraints may let a future metadata producer compute fewer components. Each proposed collapse should come with an equality to its old implementation and a consumer inventory; superficial similarity of two formulas is insufficient.

Can resource interfaces depend only on shape? A small generic structural interpretation could derive count, depth, and operation-sensitive costs from the same exact theorem. The tradeoff is API overhead. A useful next experiment is to compare the smallest shared interface against the actual remaining consumers, charging every helper and avoiding duplication of the older depth-certificate work.

Where should raw and simplified split certificates separate? Raw symmetry supplies exact equalities, whereas independent simplification supplies inequalities. A clear interface between those stages could prevent invalid rewrites and make later cost arguments shorter. The $x \cdot x$ counterexample should remain a regression example for any such interface.

Does a third residual encoding use the same monotone-window contract? Division is already prior work and cannot justify a new savings claim by itself. A genuine additional client with independently proved window bounds would determine whether the abstraction earns back its current overhead. Decoders based on disjoint shells are a natural place to inspect, but no unreviewed client is counted in this packet.

Are other new growth clients using an exponent in stages when a single application suffices? The relevant search pattern is a final term b^{ku} together with a desired linear bound in u. A full-exponent argument can sometimes remove positivity and intermediate-power lemmas. Each use must be screened against the older threshold proposals and checked at zero and boundary exponents.

Sources and artifact map

[1] Pinned ProveIt commit. Root `lean-toolchain`, `lakefile.toml`, and `lake-manifest.json` supply the compiler and dependency pins.

[2] `Expr.splitNatural`, `ExpressionSize.splitSize`, and `Expr.splitNatural_depth_le`: the exact syntax and metadata motivating A.

[3] `ObjectPairingInjective.pair_eq_pair_iff` and `ObjectDivision.division_unique`: the two mathematical interval clients. The main new patch uses packet y's `syntax/candidates/ObjectPairingInjective.lean` as its comparison baseline.

[4] `CurrentPackingRank2011.rank_gt_twice_window`: the new constructed-rank endpoint. Its introducing commit is `493d46d498`.

[5] Prior proposal collection and incoming archives. Exact evidence members, line locations, and hashes appear in `audit/prior_art_final_novelty.json`.

[6] Final 43-packet registry. The refresh includes the newly placed zp report and zq report. The source inventories in zq mention the constructed-rank theorem but do not propose its full-exponent proof.

[7] Pinned Mathlib source. The compiler is Lean 4.32.0, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`.

The archive contains this PDF and its complete LaTeX source, the three proposal directories, proof cores and logs, exact baseline/candidate source files, patch applicability evidence, an exclusion audit, and portable replay guidance. It omits repository-wide source snapshots and compiled dependency caches.